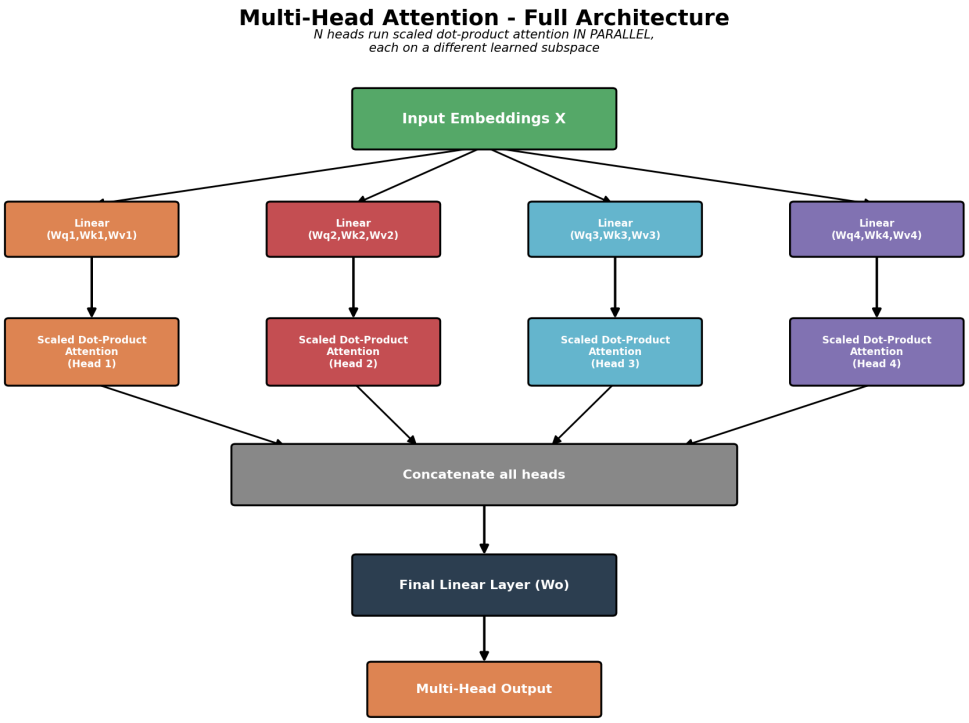
Scaled Dot-Product Attention & Multi-Head Attention

Scaling, Geometry, Visualization, and How Multiple Heads Work Together



Part 1: Why we scale • Geometric intuition • Visualizing attention • Why 'self' • Self vs Luong attention

Part 2: What is multi-head attention • Multi-head vs single-head • How it works & is applied • Visualization

PART 1 - Scaled Dot-Product Attention

1. What Is Scaled Dot-Product Attention?

This is the exact attention formula used inside every Transformer. It takes a Query, a set of Keys, and a set of Values, and returns a weighted blend of the Values - weighted by how well the Query matches each Key.

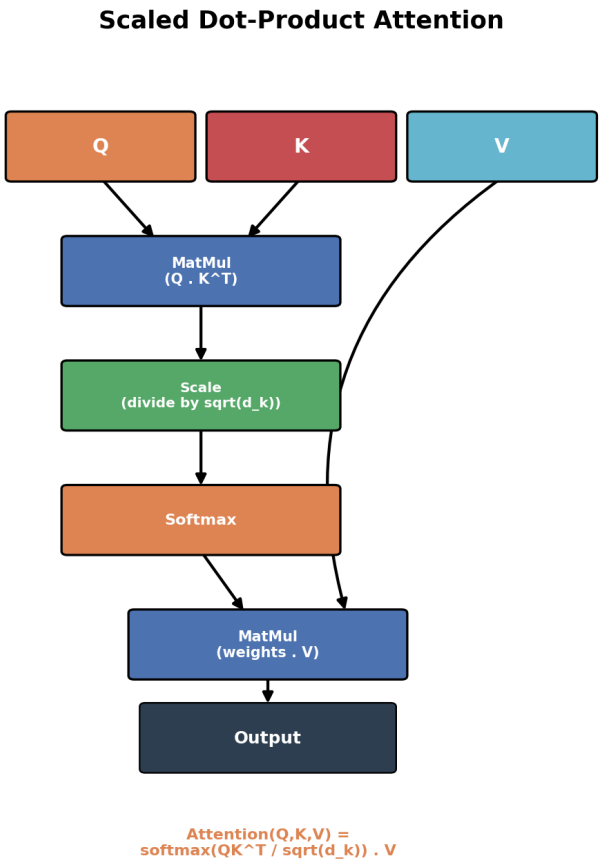


Figure 1: The scaled dot-product attention pipeline - matmul, scale, softmax, matmul.

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

2. Why Do We Scale Self-Attention?

The "scaled" part matters more than it might seem. Without dividing by sqrt(d_k) (the dimensionality of the key vectors), dot products between Q and K can grow very large in magnitude as d_k increases -

simply because you're summing more terms.

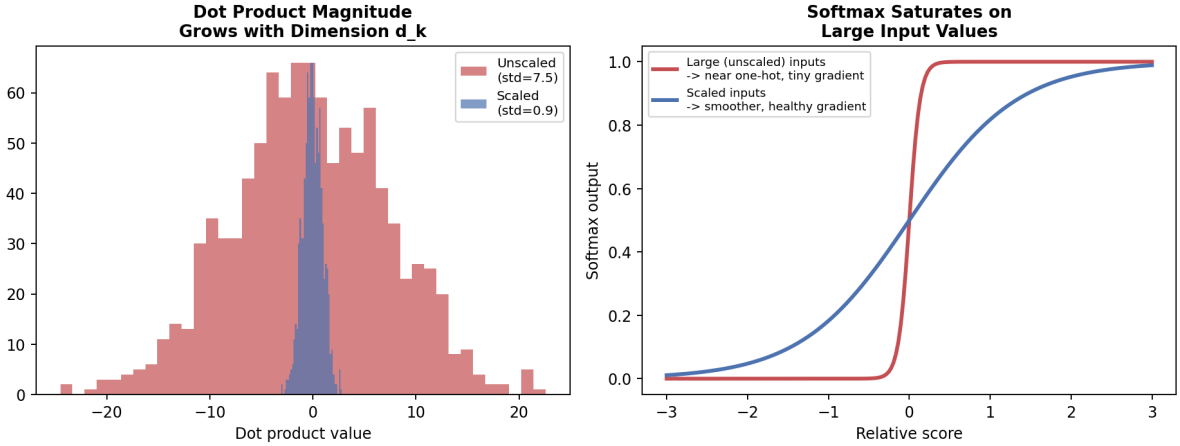


Figure 2: Left - dot products spread out more as dimensionality grows. Right - large inputs push softmax toward a near one-hot output.

Large dot-product values push the softmax function into a region where it's extremely "peaky" - almost all the probability mass goes to a single position, and the gradients for everything else become vanishingly small. This makes training unstable and slow. Dividing by sqrt(d_k) keeps the values in a numerically healthy range, so softmax produces smoother, more useful gradients.

Intuition: if Q and K have independent random entries with unit variance, their dot product has variance proportional to d_k. Dividing by sqrt(d_k) brings the variance back down to roughly 1, regardless of dimensionality.

3. Self-Attention: Geometric Intuition

Geometrically, the dot product between two vectors measures how aligned they are - vectors pointing in similar directions produce large positive dot products, while vectors pointing in different directions produce small or negative ones. This is exactly what a Query "searching" for a relevant Key is doing: measuring directional alignment in a learned vector space.

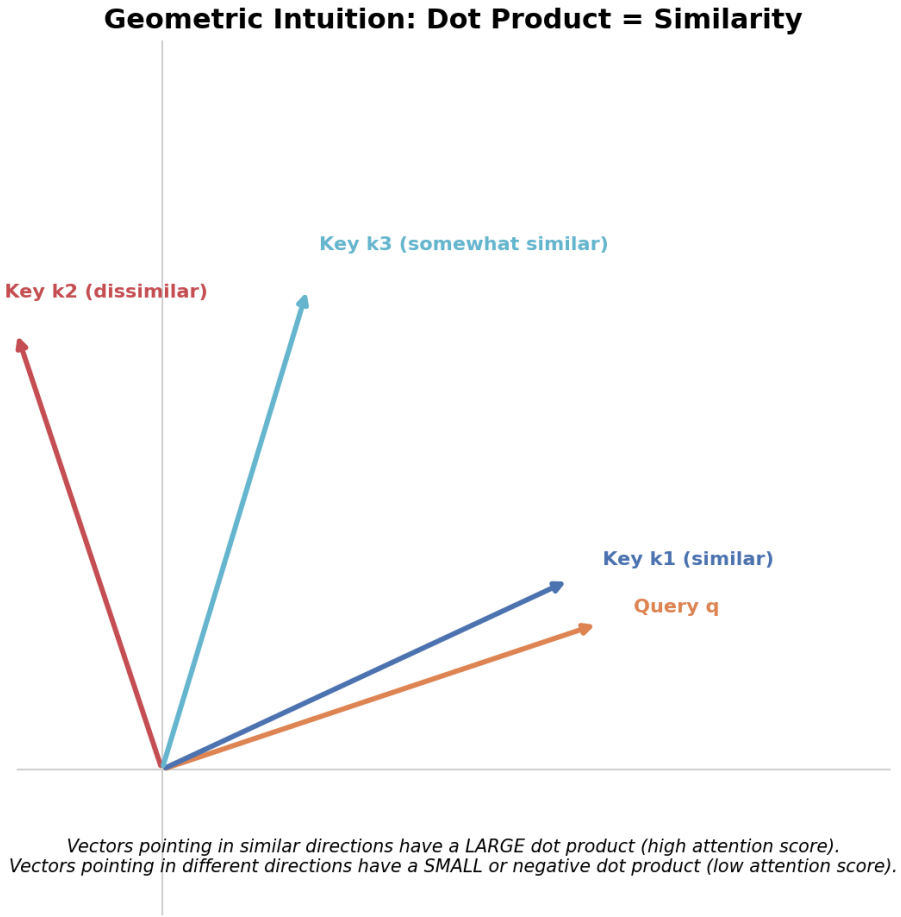


Figure 3: A Query vector is compared against several Key vectors - the more aligned they are, the higher the attention score.

4. How to Visualize Self-Attention

The most common way to visualize self-attention is as a heatmap of attention weights (rows = query words, columns = key words), or as connecting lines between words that shows which words attend most strongly to which others. Tools like BertViz popularized this style of visualization.

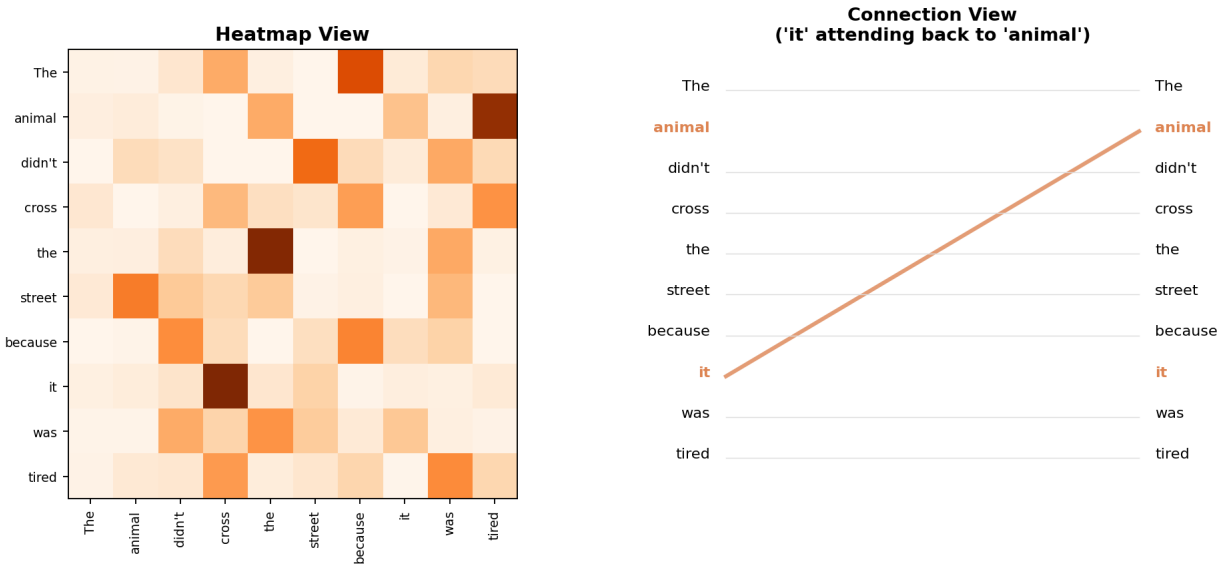


Figure 4: Two ways to visualize the same attention pattern - as a heatmap (left) or as direct word-to-word connections (right).

In the example above, the word "it" attends strongly back to "animal" - this is exactly the kind of long-range dependency resolution (pronoun/coreference resolution) that made attention so powerful compared to earlier architectures.

5. Why Is Self-Attention Called "Self"?

The name comes directly from where the Query, Key, and Value vectors are sourced from. In self-attention, ALL THREE come from the **same** input sequence - the sequence is essentially attending to itself. Compare this to **cross-attention** (used in encoder-decoder setups, like in the original Transformer's decoder), where the Query comes from one sequence (the decoder) but the Keys and Values come from a different sequence (the encoder).

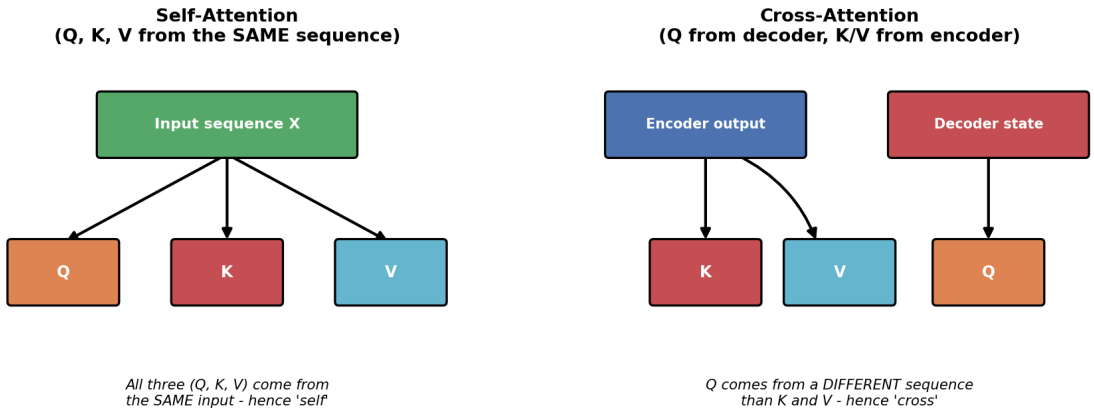


Figure 5: Self-attention draws Q, K, V from one sequence; cross-attention draws Q from a different sequence than K and V.

6. Self-Attention vs Luong Attention

Both use a similar underlying scoring idea, but they serve different roles and appear in different architectures:

Self-Attention vs Luong Attention

Aspect	Self-Attention (Transformer)	Luong Attention (Seq2Seq)
Q, K, V source	All from the SAME sequence	Q from decoder, K/V from encoder
Used within	Every layer, encoder AND decoder	Only between decoder and encoder
Recurrence needed?	No - fully parallel	Yes - built on top of an RNN/LSTM
Purpose	Relate tokens to each other within one sequence	Let decoder focus on relevant encoder positions
Score function	Scaled dot product	Dot / General / Concat

Figure 6: Self-attention (used throughout Transformers) vs Luong-style attention (used in RNN-based Seq2Seq models).

In short: Luong attention was designed to connect a decoder to an encoder within an RNN-based model. Self-attention generalizes this idea further, using it everywhere - within a single sequence, in every layer, with no recurrence required at all.

PART 2 - Multi-Head Attention

7. What Is Multi-Head Attention?

A single scaled dot-product attention computation only gives the model **one** way of relating words to each other. But language relationships are diverse - syntax, coreference, semantic similarity, positional patterns - and one attention computation can struggle to capture all of them at once. **Multi-head attention** solves this by running several independent attention computations ("heads") in parallel, each with its own learned Wq, Wk, Wv matrices, and combining their results.

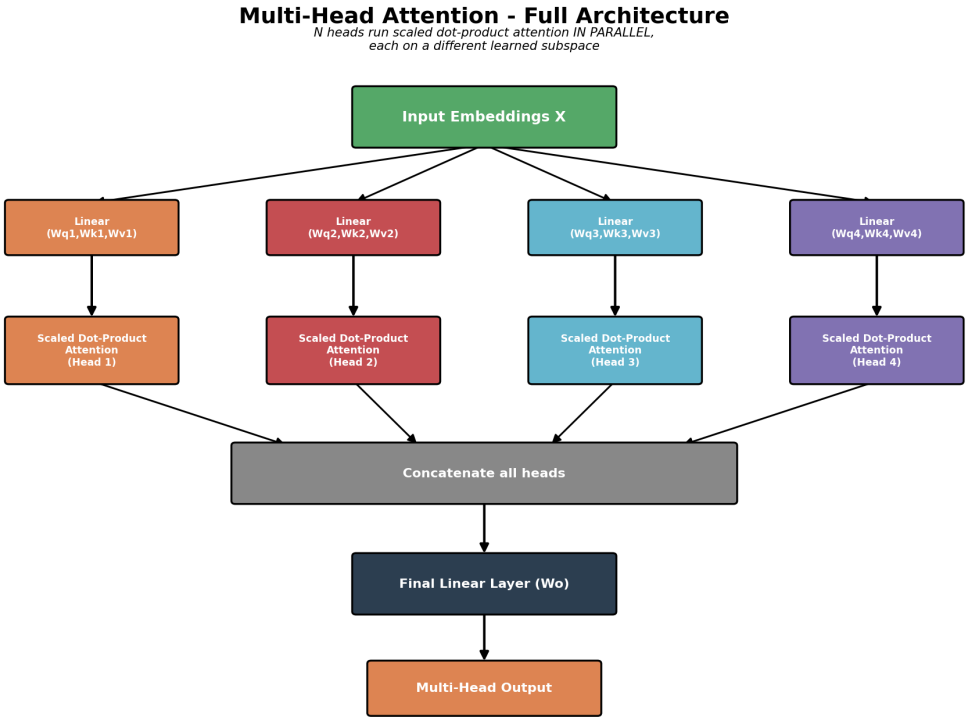


Figure 7: Multi-head attention - N independent scaled dot-product attention heads run in parallel, then their outputs are concatenated and linearly projected.

8. Multi-Head vs Single-Head Attention

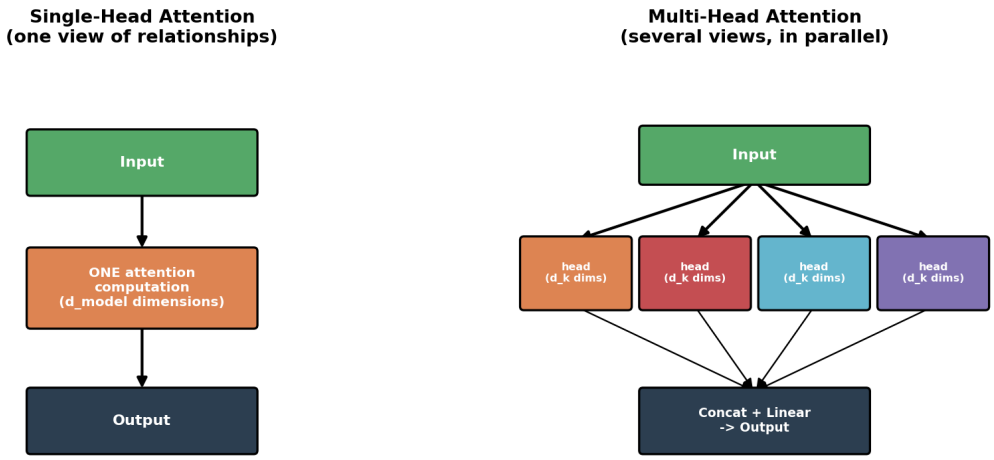


Figure 8: A single head computes one attention view in the full d_model space; multiple heads each compute attention in a smaller subspace, then combine.

Each head typically works in a reduced dimensionality (d_k = d_model / num_heads), so the total computational cost stays roughly similar to a single full-size head - but the model now has multiple independent "perspectives" to draw from.

9. How Multi-Head Attention Works & Is Applied

The full computation, step by step:

- **1. Linear projections:** the input embeddings X are projected into Q, K, V for EACH head separately, using that head's own Wq_i, Wk_i, Wv_i weight matrices.
- **2. Parallel attention:** each head independently runs scaled dot-product attention: head_i = Attention(Q_i, K_i, V_i).
- **3. Concatenate:** all head outputs are concatenated back together into one vector per position.
- **4. Final linear layer:** the concatenated vector is passed through one more learned weight matrix Wo, projecting it back to the model's original dimensionality.

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, ..., \text{head}_h) \cdot W_o, \quad \text{where head}_i = \text{Attention}(Q.W_{q_i}, K.W_{k_i}, V.W_{v_i})$$

Multi-head attention is used **everywhere** inside the Transformer: as self-attention in every encoder layer, as masked self-attention in every decoder layer, and as cross-attention connecting decoder to encoder. It's the single most reused building block in the entire architecture.

10. Visualizing Multi-Head Attention

One of the most interesting findings from analyzing trained Transformers is that different heads genuinely specialize - some focus on nearby words (local syntax), some track long-range dependencies (like coreference), some attend broadly, and some develop harder-to-interpret patterns.

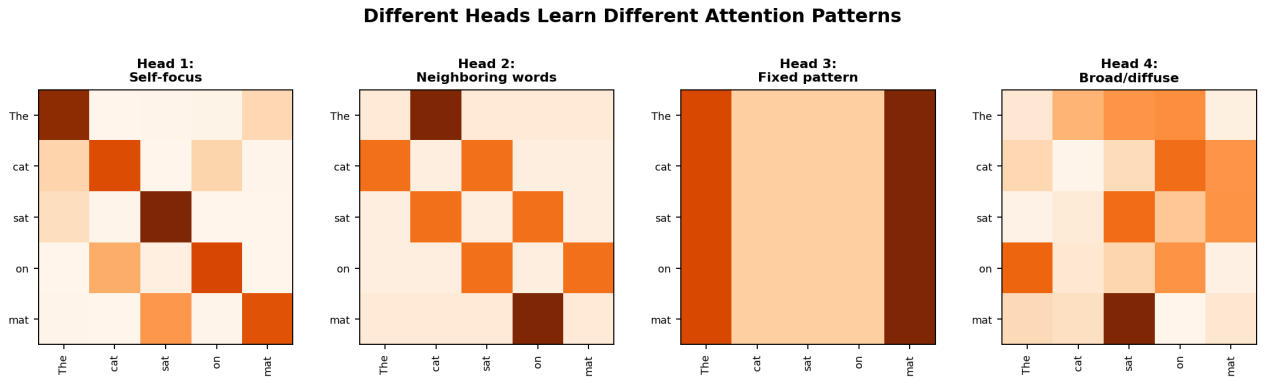


Figure 9: Four different attention heads on the same sentence, showing distinct learned patterns - self-focus, neighboring words, a fixed pattern, and broad/diffuse attention.

Summary

Concept	Key Idea
Scaled dot-product attention	softmax(QK^T / sqrt(d_k)) . V - the core attention formula
Why scale	Prevents large dot products from saturating softmax and killing gradients
Geometric intuition	Dot product measures directional alignment between Q and K vectors
Visualizing attention	Heatmaps or connection diagrams show which words attend to which
Why 'self'	Q, K, and V all come from the SAME input sequence
Self vs Luong	Self-attention works within one sequence, everywhere; Luong connects decoder to encoder in RNNs
Multi-head attention	Runs several attention computations in parallel, each its own subspace
Why multiple heads	Captures diverse relationship types (syntax, coreference, position, etc.)
Where it's used	Every encoder and decoder layer, plus cross-attention between them

Together, scaling and multi-head design are what make Transformer attention both numerically stable AND expressive enough to capture the many different kinds of relationships that make up human language.